



**Politecnico
di Torino**

Politecnico di Torino

Corso di Laurea Magistrale

A.a. 2025/2026

Software Design and Architecture

Docenti:

Antonio Vetro'
Marco Torchiano

Studente:

Cristiano Berardo

Indice

Elenco delle figure	III
1 Introduction to Software Design	1
1.1 Complexity	1
1.1.1 Syntax complexity	1
1.1.2 Cause of complexity	1
1.1.3 Strategical vs. Tactical Programming	1
1.2 Modules	1
1.2.1 Abstraction	2
1.2.2 Information Hiding	2
1.2.3 Information Leakage	2
1.2.4 Information Hiding Red Flags	2
1.2.5 Method separation	2
1.2.6 Do one simple thing and complexity	3
1.3 Obscurity	3
1.3.1 Abstraction layers	3
1.3.2 Writing Comments	3
1.3.3 Naming	3
1.4 Errors and Anomalies	4
1.4.1 Define Errors out of Existence	4
1.4.2 Limit exceptions	4
1.5 Dependency Dimensions	4
2 Design Patterns	5

Elenco delle figure

Capitolo 1

Introduction to Software Design

1.1 Complexity

Complexity is anything related to the structure of a software system that makes it hard to understand and modify the system.

$$C = \sum c_p t_p$$

Where C is determined by all software parts p , c_p is the complexity of part p , and t_p is the fraction of time spent on part p .

1.1.1 Syntax complexity

- Change amplification: change require operating in different places
- Cognitive load: how much information to learn to understand and perform a task. Some times is better to write longer code to reduce the complexity and so the cognitive load.
- Unknown unknowns: there are partes of the system that we don't know where they are or even that they exist.

1.1.2 Cause of complexity

Dependency: When we need to understand or modify a part of the code, we need to understand or modify other parts of the code. Try to reduce the number of dependencies and the complexity of the dependencies.

Obscuiry: alla the important information shoul be clearly visible.

1.1.3 Strategical vs. Tactical Programming

- **Tactical programming:** focus on having code that works as fast as possible. "Move fast and break things".
- **Strategical programming:** focus on long term maintainability and scalability. "Working code is not enough".

1.2 Modules

To improve and eliminate dependency and complexity, we can use modules. A module is a part of the code that has a well defined interface and a well defined implementation.

- **Interface:** what we need to know to use the module. Formal part, possible checked by the compiler. Represents a cost.

- **Implementation:** how the module works. Ideally much more complex than the interface. Represents a benefit.

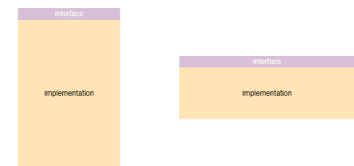
1.2.1 Abstraction

What we typically do is to perform an abstraction. Abstraction means selective removal of information. Is a simplified view of the something that omits "unimportant" information. Omitting information can lead to errors and keeping unimportant information can lead to cognitive overload.

Deep modules and shallow interfaces

Deep modules: modules with a simple interface and a complex implementation. They are good because they reduce the cognitive load and the change amplification. They are bad because they can lead to unknown unknowns.

Shallow interfaces: modules with a complex interface and a simple implementation. They are good because they reduce the unknown unknowns. They are bad because they can lead to cognitive overload and change amplification.



1.2.2 Information Hiding

Information hiding is the principle of hiding the implementation details of a module from the users of the module. Example sort function: we want to sort a list of numbers, we don't care about how the sorting algorithm used.

1.2.3 Information Leakage

Is the opposite of information hiding. Occurs when the implementation details of a module is reflected in multiple modules with consequences of creating dependencies.

1.2.4 Information Hiding Red Flags

- **temporal decomposition:** execution order is reflected in the code structure. Operations that happen in different times are in different classes.
- **overexposure:** API forces knowledge about rarely used features to use common ones. E.g.
`BufferedReader r = new BufferedReader(new InputStreamReader(System.in));`
- **Too many classes:** code is divided into too many shallow classes.

1.2.5 Method separation

Better together or Better apart?

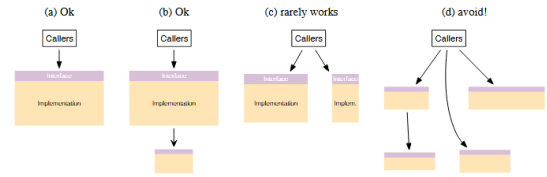
- **Bring together:** if information is shared, if we simplify the interface, if we eliminate duplication.
- **Separate:** if the code is general purpose or specific purpose.

Method separation red flags

If we have a lot of repeated code and/or we mixed specific and general code, means we have put together code that should be separated.

1.2.6 Do one simple thing and complexity

Each method should do one thing and do it completely



1.3 Obscurity

Obscurity is the principle of making the important information clearly visible and the unimportant information hidden. Obscurity is a great source of complexity. Obscure code is hard to Understand and creates more Bugs.

The solution to obscurity is writing obvious code, this means with a quick glance we can understand what the code does. We can achieve this by using abstraction layers.

1.3.1 Abstraction layers

Software systems are composed in layers. Higher layers use the facilities provided by lower layers.

Abstraction simplifies understanding and reduces obscurity.

In a well-designed system, each layer provides a different abstraction from the layers above and below it.

Abstraction Layers Red Flags

Some red flags that indicate that we have a problem with abstraction layers are:

- **Pass-Through methods:** is a method that does nothing but call another method.
- **Pass-Through variables:** is a variable that is passed through over and over down a long chain of method calls.

1.3.2 Writing Comments

Another method to reduce obscurity is writing comments. If users need to read a method code to use it, there is no abstraction.

Comments should describe things that are not obvious in the code:

- **Pick conventions:** write comments like other persons in the project
- **Don't repeat code:** avoid redundant comments that simply restate what the code already makes clear
- **Higher level comments enhance intuition**
- **Delayed comments are bad comments**
- **Use comments as design tool**

1.3.3 Naming

Toggether with comments, naming is a great tool to reduce obscurity. Bad names cause bugs. Name should create a image in mind of the reader about the nature of the thing being named. Names should be precise and consistent.

1.4 Errors and Anomalies

Exceptions add complexity because they create dependencies between the code and if they are easy to throw they are hard to handle.

A piece of code might throw exceptions in different ways:

- Caller may provide bad arguments or configuration
- Invoked method may not be able to complete the request
- In distributed systems network packets may be lost or delayed or peers may communicate in unexpected ways
- The code may detect bugs or internal inconsistencies or situations that are not prepared to handle

1.4.1 Define Errors out of Existence

What are the solutions to errors? Define errors out of existence. This means, not ignore the error but to redefine the behavior of a method so that no error is required.

E.g. `String.substring(int beginIndex, int endIndex)` it throws different exceptions in only specific cases. This is completely avoidable and unnecessary.

1.4.2 Limit exceptions

There are some methods to limit the number of exceptions that a method can throw:

- Handle exceptions at the point where they are thrown, if possible
- Exception aggregation means handle different exceptions with same handler
- Just crash
- Mask exceptions avoiding unexpected exceptions

1.5 Dependency Dimensions

1. Structural (Code-Level) Dependencies
2. Runtime / Distribution Dependencies
3. Data-Level Dependencies
4. Behavioral Dependencies

1.5.1 Structural (Code-Level) Dependencies

- **Implementation Dependency:** some piece of code depends on a concrete class instead of an abstraction
- **Construction Dependency:** you depend on how a collaboration, another class, is created.
- **Compile-Time Dependency:** the module knows something at compile time

1.5.2 Runtime / Distribution Dependencies

Often linked to distributed architectures.

- **Spatial Dependency:** A component depends on the location of another component. E.g. hard-coded URLs
- **Temporal Dependency:** A component depends on another being available at the same time. E.g. remote method calls or servers
- **Synchronization Dependency:** A component depends on another to complete work before continuing. E.g. synchronous request/response

1.5.3 Data-Level Dependencies

- **Data Dependency:** One component depends on another's internal data model. E.g. shared database
- **Schema Dependency:** Multiple services depend on the same database schema or representation structure. E.g. DB schema, XML/Json schema

1.5.4 Behavioral Dependencies

- **Control Flow Dependency:** One component dictates another's execution path. E.g. Centralized process managers
- **Transactional Dependency:** E.g. Components rely on atomic distributed consistency.
- **Timing Assumption Dependency:** Assumptions about response time or ordering. E.g. "Service must respond in <50ms" or Event ordering assumptions

Capitolo 2

Design Patterns